

Lando System Specification Language Grammar

January 19, 2022

1 Notation

The grammar for the Lando System Specification Sublanguage is written in the EBNF notation. The main elements of the notation that we utilize are:

- Keyword terminals are represented in "typewriter" font.
- Single-character terminals appear in single quotes, e.g. '!' or '\n'.
- More complex terminals are represented using standard regular expressions written as `/regexp/`, which may include escaped control characters such as `\n`. Regular expressions may use character classes (with negation). Note that the pattern `///` matches the literal character forward slash.
- An optional thing is written as `[thing]`.
- Zero or more repetitions of thing are written as `{ thing }`.
- One or more repetitions of thing are written as `{ thing }+`.
- If S and T are non-terminals or regular expressions, $S - T$ denotes the set of strings matching S but not T .
- EOF represents end-of-file; it can be thought of as a special terminal that is not consumed, but rather is replicated as often as necessary to match the grammar when parsing.
- Whitespace and line separators are significant throughout. Note that the line separator terminal $\mathcal{N}$ consumes any whitespace immediately following the actual end-of-line character(s). Two consecutive line separators (i.e. matching $\mathcal{N}\mathcal{N}$) are always lexed instead as a single blank line terminal $\mathcal{B}$; this distinction is necessary to resolve certain ambiguities in the grammar for paragraphs. The general line separator $\mathcal{L}$ is the union of $\mathcal{N}$ and $\mathcal{B}$.
- In cases of ambiguity, productions involving repetition behave “greedily,” i.e. they match as much of the input as possible. For example, because the Sentence Body production uses `{ }`, the string “`abc def ghi`” parses as a single sentence containing three words, rather than as three sentences of one word each.

1.1 Context-Free Grammar

landoSource	::=	{ $\mathcal{L}$ } { specElement } { $\mathcal{L}$ } { $\mathcal{CL}$ } { $\mathcal{L}$ } EOF	<i>Lando source</i>
specElement	::=	system subsystem component events scenarios requirements relation	<i>Specification Elements</i>
system	::=	{ $\mathcal{CL}$ } "system" elemName [abbrev] [relationClause] [$\mathcal{C}$] { $\mathcal{L}$ } ⁺ explanation [indexing blockend]	<i>System</i>
subsystem	::=	{ $\mathcal{CL}$ } "subsystem" elemName [abbrev] [relationClause] [$\mathcal{C}$] { $\mathcal{L}$ } ⁺ explanation [indexing blockend]	<i>Subsystem</i>
explanation	::=	paragraph	<i>Explanation</i>
abbrev	::=	{ $\mathcal{S}$ } ' (' { $\mathcal{S}$ } elemNameWord { $\mathcal{S}$ } ') ' { $\mathcal{S}$ }	<i>Abbreviation</i>
relationClause	::=	relationKind elemName	<i>Relation Clause</i>
relationKind	::=	"inherit" "client" "contains"	<i>Relation Kind</i>
indexing	::=	"indexing" { $\mathcal{S}$ } { { $\mathcal{L}$ } ⁺ indexEntry }	<i>Indexing</i>
indexEntry	::=	indexName ':' indexValuePart { { $\mathcal{L}$ } ⁺ indexValuePart }	<i>Index Entry</i>
indexValuePart	::=	indexName { $\mathcal{L}$ } [$\mathcal{C}$]	<i>Index Value Part</i>
component	::=	{ $\mathcal{CL}$ } "component" elemName [abbrev] [relationClause] [$\mathcal{C}$] { $\mathcal{L}$ } ⁺ explanation { componentFeature blockend }	<i>Component</i>
componentFeature	::=	constraint command query	<i>Component Feature</i>
constraint	::=	{ $\mathcal{CL}$ } sentBody ' . ' [wordSep] [$\mathcal{C}$]	<i>Constraint Feature</i>
query	::=	{ $\mathcal{CL}$ } sentBody ' ? ' [wordSep] [$\mathcal{C}$]	<i>Query Feature</i>
command	::=	{ $\mathcal{CL}$ } sentBody ' ! ' [wordSep] [$\mathcal{C}$]	<i>Command Feature</i>
events	::=	{ $\mathcal{CL}$ } "events" name [$\mathcal{C}$] { { $\mathcal{L}$ } ⁺ eventEntry } blockend	<i>Events</i>
eventEntry	::=	{ $\mathcal{CL}$ } name [$\mathcal{C}$] { $\mathcal{L}$ } ⁺ sentence [$\mathcal{C}$]	<i>Event Entry</i>
scenarios	::=	{ $\mathcal{CL}$ } "scenarios" name [$\mathcal{C}$] { { $\mathcal{L}$ } ⁺ scenarioEntry } blockend	<i>Scenarios</i>
scenarioEntry	::=	{ $\mathcal{CL}$ } name [$\mathcal{C}$] { $\mathcal{L}$ } ⁺ sentence [$\mathcal{C}$]	<i>Scenario Entry</i>
requirements	::=	{ $\mathcal{CL}$ } "requirements" name [$\mathcal{C}$] { { $\mathcal{L}$ } ⁺ requirementEntry } blockend	<i>Requirements</i>
requirementEntry	::=	{ $\mathcal{CL}$ } name [$\mathcal{C}$] { $\mathcal{L}$ } ⁺ sentence [$\mathcal{C}$]	<i>Requirement Entry</i>
relation	::=	{ $\mathcal{CL}$ } "relation" elemName [relationClause] [$\mathcal{C}$] blockend	<i>Relation Declaration</i>
name	::=	{ $\mathcal{S}$ } nameWord { { $\mathcal{S}$ } ⁺ nameWord } { $\mathcal{S}$ }	<i>Name</i>
nameWord	::=	/[^\\r\\n \\t()]*[^\\r\\n \\t().!?!?]/ - ///.*/	<i>Name word</i>
indexName	::=	{ $\mathcal{S}$ } indexNameWord { { $\mathcal{S}$ } ⁺ indexNameWord } { $\mathcal{S}$ }	<i>Index name</i>
indexNameWord	::=	/[^\\r\\n \\t:]+/ - ///.*/	<i>Index name word</i>
elemName	::=	{ $\mathcal{S}$ } elemNameWord { { $\mathcal{S}$ } ⁺ elemNameWord } { $\mathcal{S}$ }	<i>Element name</i>
elemNameWord	::=	nameWord - relationKind	<i>Element name word</i>
paragraph	::=	{ sentence } ⁺ parend	<i>Paragraph</i>
parend	::=	$\mathcal{B}$ { $\mathcal{L}$ } EOF	<i>Paragraph end</i>
sentence	::=	sentBody [sentTerm] [wordSep]	<i>Sentence</i>
sentBody	::=	sentWord { wordSep sentWord } [wordSep]	<i>Sentence Body</i>
wordSep	::=	{ $\mathcal{S}$ } ⁺ { $\mathcal{S}$ } $\mathcal{L}$	<i>Word Separator</i>
sentWord	::=	/[^\\r\\n \\t]*[^\\r\\n \\t().!?!?]/ - ///.*/	<i>Sentence word</i>
sentTerm	::=	' . ' ' ! ' ' ? '	<i>Sentence terminator</i>
blockend	::=	{ $\mathcal{L}$ } ⁺ EOF	<i>Block end</i>
$\mathcal{S}$	::=	/[\\t]/	<i>Whitespace character</i>
$\mathcal{N}$	::=	['\\r'] '\\n' [$\mathcal{S}$] '\\r' [$\mathcal{S}$] EOF	<i>Single line separator</i>
$\mathcal{B}$	::=	$\mathcal{N}\mathcal{N}$	<i>Blank line</i>
$\mathcal{L}$	::=	$\mathcal{N}$ $\mathcal{B}$	<i>General line separator</i>
$\mathcal{C}$	::=	///[^\\r\\n]*	<i>Comment</i>

2 Context-Sensitive Considerations

The following extra-grammatical rules are applied during or after parsing the concrete grammar above.

- An implicit containment relationship is recorded from each subsystem into the system, if any, most recently defined above it.
- An implicit containment relationship is recorded from each component into the subsystem, if any, most recently defined above it.
- Each relationClause is translated into an equivalent top-level relation declaration.
- A sentence that does not end with a '.', '!', or '?' symbol (possibly followed by white space characters or line separators) induces a warning message.
- Note that because of the “greedy” resolution of ambiguity, this warning can only occur at the end of a paragraph.